R Control Flow Statements: Building Blocks for Automated Decision-Making

Sherine Paul Arthur

Fresenius University of Applied Science | 401147341

Data Analysis for Decision-Making (SS 2026)

Prof. Dr. Stephan Huber

2026-05-19

Author Note

Correspondence concerning this article should be addressed to Sherine Paul Arthur, Email: paul_arthur.sherine@stud.hs-fresenius.de

Abstract

Every useful R program makes decisions. It checks whether a value meets a condition, repeats a calculation across a dataset, or skips rows that do not belong. These behaviours are not built into R automatically: you write them using control flow statements. This handout is a practical reference for the control flow tools covered in the presentation: if/else, ifelse(), for loops, while loops, break, and next. Each section explains the concept, shows the syntax, and demonstrates it with a realistic example.

R Control Flow Statements: Building Blocks for Automated Decision-Making

1 What is Control Flow?

When you read a page you go from top to bottom, by line. R does exactly this too! When you're new with R you will find that this is fine because the built in packages already perform most of what you need. but as you progress you will want to work with real data and do some of the coding yourself. so this is not enough.

Real data analysis requires decisions. A value might be missing. A column might need to be recoded. A loop might need to stop early. You cannot anticipate all of this in advance, because the answers depend on what the data actually contains. Control flow is how you write code that responds to those conditions rather than ignoring them.

According to Lenovo (n.d.), control flow refers to the order in which instructions in a program are executed, including how that order changes based on conditions and logic. In R, there are three ways to control that order:

- **Branching** — run a block of code only when a condition is met
- **Looping** — repeat a block of code over a sequence or until a condition changes
- **Jumping** — interrupt a loop mid-run

Lets go over this one by one.

2 Conditional Statements

2.1 if, else if, and else

An `if` statement evaluates a condition and runs a block of code only if that condition is `TRUE`. If it is `FALSE`, R skips the block and moves on. The `else` clause provides an alternative — what to do when the condition is not met.

```
score <- 85

if (score >= 90) {
  print("Distinction")
} else if (score >= 60) {
  print("Pass")
} else {
  print("Fail")
}
```

```
[1] "Pass"
```

Note: R checks each condition from top to bottom and stops at the first one that is `TRUE`. Everything below it is skipped. This matters for how you order your conditions so always start with the most specific case and work toward the most general. If you put the broadest condition first, the more specific ones will never be reached.

2.2 The == Operator

A common mistake that occurs here while coding in R is using = instead of == inside a condition. The single = assigns a value to a variable. The double == tests whether two values are equal. They look almost identical but do completely different things (Peng, 2020).

```
status <- "active"

if (status == "active") {
  print("Account is active")
}
```

```
[1] "Account is active"
```

When in doubt: assignment uses <-, comparison uses ==.

2.3 ifelse() for Vectors

The `if / else` structure works on a single value at a time. In data analysis, you routinely need to apply a condition to an entire column. The `ifelse()` function does exactly this — it evaluates the condition for every element of a vector and returns a result for each one.

```
# Final exam results
students <- c("Ali", "Luisa", "Roopam", "Mahdi", "Cagan")
scores   <- c(82, 47, 91, 55, 68)

result <- ifelse(scores >= 60, "Pass", "Fail")

for (i in 1:length(students)) {
  cat(students[i], "→", result[i], "\n")
}
```

```
Ali → Pass
Luisa → Fail
Roopam → Pass
Mahdi → Fail
Cagan → Pass
```

`ifelse()` is one of the most practical tools in data cleaning. It can recode variables, flag outliers, and create new columns based on existing ones and it does it all in a single line (Wickham, 2019).

3 Loops

3.1 for Loops

A `for` loop runs a block of code once for every item in a sequence. The loop variable takes each value in turn, and when the sequence is exhausted, the loop stops automatically.

```
months <- c("January", "February", "March")

for (m in months) {
  cat("Checking data for", m, "\n")
}
```

```
Checking data for January
Checking data for February
Checking data for March
```

A common real-world use case is applying the same operation to every column in a data frame.

```
# Weekly coffee budget per student (€)
students <- c("Sherine", "Milad", "Nwanneka", "Mohammed")
budget   <- c(18, 22, 9, 31)

for (i in 1:length(students)) {
  if (budget[i] > 25) {
    cat(students[i], "- needs an intervention\n")
  } else {
    cat(students[i], "- doing fine\n")
  }
}
```

```
Sherine - doing fine
Milad - doing fine
Nwanneka - doing fine
Mohammed - needs an intervention
```

3.2 while Loops

A `for` loop is appropriate when you know what you are iterating over. A `while` loop is appropriate when you know when to stop, but not how many steps it will take to get there.

```
balance <- 500
month   <- 0

while (balance < 1000) {
  balance <- balance * 1.05
  month   <- month + 1
}

cat("Reached €1000 after", month, "months\n")
```

```
Reached €1000 after 15 months
```

The loop keeps running as long as `balance < 1000` is `TRUE`. Once the balance exceeds 1000, the condition becomes `FALSE` and the loop stops.

The critical rule with `while` loops is that something inside the loop must move the program toward the exit condition. If `balance` never changed, the condition would remain `TRUE` forever, this is an infinite loop, and R will freeze until you force-quit it (R Core Team, 2024).

4 Loop Control: break and next

Two keywords give you finer control over what happens inside a loop.

4.1 break

`break` exits the loop immediately. Any remaining iterations are cancelled.

```
readings <- c(4.2, 3.8, 5.1, -99.0, 4.7, 3.9)

for (r in readings) {
  if (r < 0) {
    cat("Invalid reading detected:", r, "- stopping.\n")
    break
  }
  cat("Reading:", r, "\n")
}
```

```
Reading: 4.2
```

```
Reading: 3.8
Reading: 5.1
Invalid reading detected: -99 - stopping.
```

Use `break` when a certain condition means there is no point continuing. for instance, when a data quality check fails and further processing would produce meaningless results.(Hamelg, 2021)

4.2 next

`next` skips the current iteration and moves on to the next one. The loop continues normally.

```
readings <- c(4.2, 3.8, -99.0, 5.1, -99.0, 3.9)

for (r in readings) {
  if (r < 0) next
  cat("Reading:", r, "\n")
}
```

```
Reading: 4.2
Reading: 3.8
Reading: 5.1
Reading: 3.9
```

The difference between `break` and `next` is straightforward: `break` ends the loop entirely, `next` skips one step and keeps going.

5 Summary

Table 1 summarises the control flow tools covered in this handout.

Table 1
Summary of R control flow tools

Statement	What it does	When to use it
`if / else`	Runs code when a condition is true	Single-value decisions
`else if`	Adds additional branches	More than two outcomes
`ifelse()`	Applies a condition to a whole vector	Column-wise transformations
`for`	Repeats over a known sequence	Columns, rows, file lists
`while`	Repeats until a condition changes	Unknown number of iterations
`break`	Exits the loop immediately	Stop on a specific condition
`next`	Skips one iteration	Filter values mid-loop

6 References

Hamelg. (2021). *Intro to R part 11: Control flow*. https://www.kaggle.com/code/hamelg/intro-to-r-part-11-control-flow

Lenovo. (n.d.). *Control flow*. https://www.lenovo.com/us/en/glossary/control-flow/

Peng, R. D. (2020). *R programming for data science*. Leanpub. https://bookdown.org/rdpeng/rprogdatascience/

R Core Team. (2024). *R: A language and environment for statistical computing*. R Foundation for Statistical Computing. https://www.R-project.org/

Wickham, H. (2019). *Advanced R* (2nd ed.). Chapman; Hall/CRC. https://adv-r.hadley.nz/

Appendix

Affidavit

I hereby affirm that this submitted paper was authored unaided and solely by me. Additionally, no other sources than those in the reference list were used. Parts of this paper, including tables and figures, that have been taken either verbatim or analogously from other works have in each case been properly cited with regard to their origin and authorship. This paper either in parts or in its entirety, be it in the same or similar form, has not been submitted to any other examination board and has not been published.

I acknowledge that the university may use plagiarism detection software to check my thesis. I agree to cooperate with any investigation of suspected plagiarism and to provide any additional information or evidence requested by the university.

- ☒ The handout contains 3–5 pages of text.
- ☒ The submission contains the Quarto file of the handout.
- ☒ The submission contains the Quarto file of the presentation.
- ☒ The submission contains the HTML file of the handout.
- ☒ The submission contains the HTML file of the presentation.
- ☒ The submission contains the PDF file of the handout.
- ☒ The submission contains the PDF file of the presentation.
- ☒ The title page contains name, email, and matriculation number.
- ☒ The handout contains a bibliography in APA citation style.
- ☒ R code is included.
- ☒ The exercise is included.
- ☒ The affidavit is filled out.
- ☒ Link to presentation published on GitHub.
- ☒ Link to handout published on GitHub.

Sherine Paul Arthur | 401147341 | 2026-05-19 | Cologne